

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

Bài tập lớn môn: An toàn bảo mật thông tin

Đề tài: Phát triển giao thức bảo mật ứng dụng đảm bảo quyền riêng tư trong phát hiện tấn công Web

Nhóm 7:	Lê Trần Duy Luân Hoàng Hải Long Nguyễn Thanh Thiên
Lớp:	68IT4
Giảng viên hướng dẫn:	Th.S Nguyễn Việt Nhật

❖ Giới thiệu:

Giao thức **HTTP (Hypertext Transfer Protocol)** và **HTTPS (HTTP Secure)** là nền tảng cho việc truyền tải thông tin trên World Wide Web¹ Dưới đây là trình bày chi tiết về hoạt động, ưu điểm và nhược điểm của từng giao thức dựa trên các nguồn tài liệu được cung cấp:

1. Giao thức HTTP (Hypertext Transfer Protocol)

• Hoạt động:

- HTTP là một giao thức **phi trạng thái (stateless)** ở **tầng ứng dụng (application-level)**, hoạt động theo mô hình **yêu cầu/phản hồi (request/response)**³
- Một chương trình đóng vai trò là **máy khách (HTTP client)** sẽ thiết lập một **kết nối (connection)** đến một chương trình đóng vai trò là **máy chủ (HTTP server)**⁶ . Mục đích của kết nối này là để máy khách gửi một hoặc nhiều **yêu cầu HTTP (HTTP requests)**⁶ .
- Máy chủ HTTP sẽ **chấp nhận (accept)** các kết nối để **phục vụ (service)** các yêu cầu HTTP bằng cách gửi lại **phản hồi HTTP (HTTP responses)**⁶ .
- **Tài nguyên (resource)** là mục tiêu của một yêu cầu HTTP và thường được xác định bằng **định danh tài nguyên thống nhất (Uniform Resource Identifier - URI)**⁷
- Giao tiếp HTTP chủ yếu bao gồm việc máy khách gửi một yêu cầu **GET** để lấy một **biểu diễn (representation)** của một tài nguyên⁹ Biểu diễn này là thông tin phản ánh trạng thái của tài nguyên tại một thời điểm nhất định¹¹ .
- Các thông điệp HTTP bao gồm **dữ liệu điều khiển (control data)** ở đầu (ví dụ: phương thức yêu cầu, URI mục tiêu, phiên bản giao thức cho yêu cầu; mã trạng thái, cụm từ lý do, phiên bản giao thức cho phản hồi) và theo sau là các **trường (fields)** trong phần header và trailer của thông điệp¹² Các trường này cung cấp thông tin metadata về thông điệp¹³ . Thông điệp cũng có thể chứa **dữ liệu biểu diễn (representation data)**¹⁰ .
- HTTP được thiết kế để **ẩn thông tin (information hiding)** về cách một dịch vụ được triển khai bằng cách cung cấp một **giao diện (interface)** **đồng nhất** cho máy khách, độc lập với loại tài nguyên được cung cấp⁴ .
- Máy chủ không cần biết mục đích cụ thể của từng máy khách; mỗi yêu cầu có thể được xem xét độc lập⁴ .
- HTTP cũng được thiết kế để hoạt động như một giao thức **trung gian (intermediation protocol)**, cho phép các **proxy** và **gateway** chuyển đổi các hệ thống thông tin không phải HTTP thành một giao diện chung⁹

◦ HTTP đã trải qua nhiều giai đoạn phát triển với các phiên bản khác nhau như HTTP/0.9, HTTP/1.0, HTTP/1.1, HTTP/2 và HTTP/3 Tất cả các phiên bản chính này đều dựa trên các **ngữ nghĩa (semantics)** cốt lõi được định nghĩa trong tài liệu RFC 9110 này¹⁶

• Ưu điểm:

- **Đơn giản (simple):** Giao thức request/response cơ bản rất dễ hiểu và triển khai¹ .
- **Linh hoạt (flexible):** HTTP hỗ trợ nhiều loại tài nguyên khác nhau và có thể được mở rộng thông qua việc định nghĩa các phương thức mới, mã trạng thái mới và các trường header mới⁴
- **Phi trạng thái (stateless):** Mỗi yêu cầu được xử lý độc lập, không phụ thuộc vào các yêu cầu trước đó, giúp máy chủ dễ dàng quản lý và mở rộng⁴
- **Khả năng tương tác (interoperable):** HTTP là một tiêu chuẩn mở, cho phép các hệ thống khác nhau có thể giao tiếp với nhau một cách dễ dàng⁴ .
- **Khả năng mở rộng (extensible):** Giao thức được thiết kế để có thể phát triển theo thời gian với các phiên bản và phần mở rộng mới mà không làm ảnh hưởng đến các chức năng cốt lõi¹
- **Sử dụng rộng rãi (widely used):** HTTP là giao thức truyền tải thông tin chính cho World Wide Web kể từ khi ra đời vào năm 1990¹ .

• Nhược điểm:

- **Bảo mật (security):** HTTP truyền dữ liệu **không được mã hóa (unencrypted)**¹⁹ Điều này có nghĩa là dữ liệu có thể bị chặn và đọc bởi các bên thứ ba (ví dụ: tấn công nghe lén - eavesdropping). Thông tin nhạy cảm như mật khẩu, thông tin cá nhân có thể bị đánh cắp.
- **Phi trạng thái (stateless):** Mặc dù là một ưu điểm về khả năng mở rộng, tính phi trạng thái đôi khi gây khó khăn trong việc duy trì thông tin người dùng giữa các yêu cầu khác nhau. Các cơ chế bổ sung như **cookies** thường được sử dụng để giải quyết vấn đề này⁴
- **Hiệu suất (performance) (HTTP/1.1 trở về trước):** Với các phiên bản HTTP/1.1 trở về trước, mỗi yêu cầu và phản hồi thường cần một kết nối TCP riêng biệt. Việc thiết lập và đóng các kết nối này có thể gây ra độ trễ và làm giảm hiệu suất, đặc biệt với các trang web chứa nhiều tài nguyên nhỏ¹⁵ Các phiên bản mới hơn như HTTP/2 và HTTP/3 đã giới thiệu các kỹ thuật như **multiplexing** để truyền nhiều yêu cầu và phản hồi trên cùng một kết nối, giúp cải thiện hiệu suất¹⁶ .

2. Giao thức HTTPS (HTTP Secure)

• Hoạt động:

- `HTTPS không phải là một giao thức riêng biệt mà là **HTTP over TLS (Transport Layer Security)**³ ... (hoặc SSL - Secure Sockets Layer, tiền thân của TLS, dù ngày nay TLS được sử dụng phổ biến hơn). Điều này có nghĩa là HTTP hoạt động trên một lớp bảo mật được cung cấp bởi TLS/SSL.
- HTTPS sử dụng **lược đồ URI (URI scheme)** là "**https**"²

- Theo mặc định, HTTPS sử dụng **cổng TCP (TCP port)** là **44324** , khác với HTTP thông thường sử dụng cổng 8025 .

- Khi một máy khách muốn truy cập một tài nguyên thông qua HTTPS, quá trình giao tiếp sẽ bắt đầu bằng việc thiết lập một **kết nối TCP** đến máy chủ²⁶ . Sau đó, máy khách và máy chủ sẽ trải qua một quá trình gọi là **bắt tay TLS (TLS handshake)**²³

- Mục đích của TLS handshake là để:

- **Xác thực (authenticate)** máy chủ (thường thông qua việc máy chủ cung cấp **chứng chỉ số (digital certificate)**)²³ Chứng chỉ này được phát hành bởi một tổ chức chứng thực (Certificate Authority - CA) và đảm bảo danh tính của máy chủ.

- **Thiết lập một kênh liên lạc được mã hóa (establish an encrypted communication channel)** bằng cách thương lượng một bộ mã hóa (cipher suite) mà cả máy khách và máy chủ đều hỗ trợ²³

- Khi kết nối TLS đã được thiết lập thành công, tất cả dữ liệu HTTP (yêu cầu và phản hồi) sẽ được **mã hóa (encrypted)** trước khi được truyền đi và **giải mã (decrypted)** khi đến đích²³ Điều này đảm bảo **tính bảo mật (confidentiality)** của dữ liệu.

- TLS cũng cung cấp **bảo vệ tính toàn vẹn (integrity protection)**, đảm bảo rằng dữ liệu không bị thay đổi trong quá trình truyền²³ . Nếu dữ liệu bị can thiệp, các bên giao tiếp có thể phát hiện ra điều này.

- Máy khách phải **xác minh (verify)** rằng danh tính của dịch vụ (thông qua chứng chỉ) phù hợp với **máy chủ gốc (origin server)** của URI²⁷ . Quá trình xác minh chứng chỉ này rất quan trọng để ngăn chặn các cuộc tấn công **giả mạo máy chủ (server impersonation)**²⁷

- Các tài nguyên được cung cấp thông qua lược đồ "**https**" được coi là có **nguồn gốc (origin)** riêng biệt so với lược đồ "**http**", ngay cả khi chúng được kiểm soát bởi cùng một thực thể¹⁹

- **Ưu điểm:**

- **Bảo mật (security):** Ưu điểm chính của HTTPS là cung cấp **mã hóa đầu cuối (end-to-end encryption)** cho dữ liệu HTTP thông qua TLS/SSL²³ Điều này bảo vệ thông tin nhạy cảm khỏi bị nghe lén và đánh cắp.

- **Xác thực máy chủ (server authentication):** Thông qua chứng chỉ số, HTTPS giúp máy khách xác minh được danh tính của máy chủ mà họ đang kết nối, giảm nguy cơ bị tấn công bởi các trang web giả mạo²³

- **Bảo vệ tính toàn vẹn dữ liệu (data integrity protection):** TLS/SSL đảm bảo rằng dữ liệu không bị thay đổi hoặc can thiệp trong quá trình truyền tải²³ .

- **Tăng độ tin cậy (increased trust):** Các trang web sử dụng HTTPS thường được người dùng tin tưởng hơn do có biểu tượng khóa bảo mật trên trình duyệt.

- **Nhược điểm:**

- **Hiệu suất (performance):** Việc thiết lập kết nối HTTPS đòi hỏi thêm thời gian cho quá trình TLS handshake so với kết nối HTTP thông thường²⁶ Quá trình mã hóa và giải mã dữ liệu cũng tiêu tốn tài nguyên CPU trên cả máy khách và máy chủ, có thể làm tăng độ trễ

và giảm hiệu suất²⁶ Tuy nhiên, với sự phát triển của phần cứng và các giao thức mới như HTTP/2 và HTTP/3, sự khác biệt về hiệu suất thường không đáng kể trong nhiều trường hợp.

- **Chi phí (cost):** Việc có được và duy trì chứng chỉ số, đặc biệt là các chứng chỉ có giá trị xác thực cao hơn, có thể tốn kém.

- **Độ phức tạp (complexity):** Việc cấu hình và quản lý chứng chỉ TLS/SSL trên máy chủ có thể phức tạp hơn so với việc chỉ sử dụng HTTP.

❖ **Đánh giá và nhận xét:**

RFC 9110 định nghĩa giao thức HTTP và cũng đề cập đến lược đồ URI "https" như một phần của kiến trúc HTTP1 . Tài liệu này cung cấp định nghĩa chính thức về lược đồ URI "https", nhấn mạnh vào việc **bảo mật kết nối thông qua TLS** để đảm bảo **tính bảo mật (confidentiality) và toàn vẹn (integrity)** của giao tiếp HTTP2 . RFC 9110 làm rõ rằng tài nguyên có sẵn thông qua lược đồ "https" không có chung danh tính với lược đồ "http", mà là các nguồn gốc riêng biệt với không gian tên riêng³ . Tài liệu cũng nhấn mạnh tầm quan trọng của việc **xác thực máy chủ thông qua chứng chỉ** trong quá trình thiết lập kết nối "https"² Nhìn chung, RFC 9110 cung cấp một nền tảng ngữ nghĩa cho HTTPS trong bối cảnh kiến trúc tổng thể của HTTP.

- **Tóm tắt những gì rút ra được:**

- **HTTPS là HTTP được truyền qua kết nối TLS đã được bảo mật²** . Lược đồ URI cho HTTPS là "https"²
- Máy chủ cho URI "https" lắng nghe kết nối TCP trên cổng 443 theo mặc định và có khả năng thiết lập kết nối TLS đã được bảo mật cho giao tiếp HTTP⁷ .
- Kết nối "https" được coi là "**secured**", nghĩa là máy chủ đã được xác thực là đang hành động thay mặt cho authority được xác định và tất cả giao tiếp HTTP với máy chủ đó đều có sự bảo vệ về tính bảo mật và tính toàn vẹn mà cả máy khách và máy chủ đều chấp nhận² .
- Máy khách **phải đảm bảo rằng các yêu cầu HTTP cho tài nguyên "https" được bảo mật** trước khi được truyền đi và chỉ chấp nhận các phản hồi được bảo mật cho các yêu cầu đó⁸ .
- Để truy cập tài nguyên "https", máy khách có thể cố gắng giải quyết host identifier thành địa chỉ IP, thiết lập kết nối TCP, bảo mật kết nối end-to-end bằng cách khởi tạo TLS thành công và gửi yêu cầu HTTP chứa request target phù hợp⁹ . Phản hồi không tạm thời từ máy chủ được coi là câu trả lời có thẩm quyền⁵ .
- **Xác minh chứng chỉ là bắt buộc** để ngăn chặn mạo danh máy chủ⁵ . Máy khách phải xác minh rằng danh tính của dịch vụ khớp với origin server của URI⁵ .

- **Lợi ích thực tế của HTTPS trong đời sống:**

Dựa trên định nghĩa và các yếu tố bảo mật được nhấn mạnh trong RFC 9110, lợi ích thực tế của HTTPS trong đời sống bao gồm:

- **Bảo vệ dữ liệu cá nhân:** HTTPS mã hóa dữ liệu được truyền giữa người dùng và máy chủ² , bao gồm thông tin đăng nhập, chi tiết thanh toán và các dữ liệu nhạy cảm khác. Điều

này ngăn chặn kẻ xấu nghe lén và đánh cắp thông tin này, đặc biệt quan trọng trong các giao dịch trực tuyến, ngân hàng điện tử và truyền thông cá nhân.

- **Đảm bảo tính toàn vẹn của dữ liệu:** HTTPS đảm bảo rằng dữ liệu không bị giả mạo hoặc thay đổi trong quá trình truyền2 . Điều này rất quan trọng để người dùng có thể tin tưởng vào thông tin họ nhận được từ các trang web, đặc biệt là các thông tin liên quan đến giao dịch tài chính hoặc các vấn đề quan trọng khác.

- **Xác thực trang web:** Chứng chỉ TLS/SSL được sử dụng trong HTTPS giúp xác thực danh tính của máy chủ2 Người dùng có thể tin tưởng hơn vào các trang web sử dụng HTTPS vì họ có thể xác minh rằng họ đang giao tiếp với đúng máy chủ hợp pháp và không phải là một trang web giả mạo (phishing)5

- **Tăng cường niềm tin của người dùng:** Khi người dùng thấy biểu tượng khóa hoặc "https" trong thanh địa chỉ trình duyệt, họ sẽ cảm thấy an tâm hơn khi tương tác với trang web, chia sẻ thông tin cá nhân hoặc thực hiện giao dịch. Điều này đặc biệt quan trọng đối với các trang web thương mại điện tử và các dịch vụ trực tuyến yêu cầu người dùng cung cấp thông tin.

- **Cải thiện thứ hạng SEO:** Mặc dù không được đề cập trực tiếp trong RFC 9110, nhưng HTTPS là một yếu tố xếp hạng nhỏ nhưng ngày càng quan trọng trong các thuật toán tìm kiếm của các công cụ tìm kiếm lớn. Việc sử dụng HTTPS có thể mang lại lợi thế nhỏ trong việc cải thiện thứ hạng trang web.

- Một số khó khăn/thách thức trong việc triển khai HTTPS:

Mặc dù RFC 9110 không đi sâu vào các thách thức triển khai, dựa trên sự hiểu biết chung về HTTPS, có một số khó khăn và thách thức trong việc triển khai HTTPS:

- **Chi phí chứng chỉ:** Mặc dù hiện nay có các tổ chức cung cấp chứng chỉ TLS/SSL miễn phí (ví dụ: Let's Encrypt), việc mua và quản lý chứng chỉ từ các nhà cung cấp thương mại có thể tốn kém, đặc biệt đối với các tổ chức lớn với nhiều tên miền và máy chủ.

- **Phức tạp trong cấu hình:** Việc cấu hình HTTPS trên máy chủ web có thể phức tạp, đòi hỏi kiến thức về TLS/SSL, cấu hình máy chủ và quản lý chứng chỉ. Cấu hình sai có thể dẫn đến các vấn đề bảo mật hoặc gián đoạn dịch vụ.

- **Vấn đề về hiệu suất:** Mặc dù sự khác biệt về hiệu suất giữa HTTP và HTTPS đã giảm đáng kể trong những năm gần đây, quá trình mã hóa và giải mã TLS/SSL vẫn có thể gây ra một chút overhead, đặc biệt đối với các trang web có lưu lượng truy cập rất lớn.

- **Quản lý và gia hạn chứng chỉ:** Chứng chỉ TLS/SSL có thời hạn nhất định và cần được gia hạn định kỳ. Việc quên gia hạn có thể dẫn đến việc trình duyệt cảnh báo về kết nối không an toàn, gây ảnh hưởng đến uy tín và trải nghiệm người dùng.

- **Các vấn đề tương thích:** Trong một số trường hợp hiếm hoi, có thể xảy ra các vấn đề tương thích giữa các phiên bản TLS/SSL khác nhau hoặc giữa các trình duyệt và máy chủ cũ hơn. Tuy nhiên, các tiêu chuẩn hiện đại thường cố gắng đảm bảo tính tương thích ngược.

- **Tấn công MITM phức tạp hơn nhưng vẫn có thể xảy ra:** Mặc dù HTTPS cung cấp một lớp bảo vệ mạnh mẽ, các cuộc tấn công Man-in-the-Middle (MITM) vẫn có thể xảy ra nếu kẻ tấn công có thể xâm nhập và kiểm soát quá trình thiết lập kết nối TLS hoặc đánh lừa người dùng cài đặt chứng chỉ độc hại. Tuy nhiên, HTTPS làm cho các cuộc tấn công này trở nên khó khăn hơn nhiều so với HTTP.

Tóm lại, HTTPS là một phần không thể thiếu của web hiện đại, mang lại nhiều lợi ích quan trọng về bảo mật và tin cậy. Mặc dù có một số thách thức trong việc triển khai, những lợi ích mà HTTPS mang lại thường vượt xa những khó khăn này, và việc áp dụng HTTPS ngày càng trở nên phổ biến và được khuyến khích rộng rãi.

❖ KẾT LUẬN

Trong khuôn khổ báo cáo này, chúng tôi đã tìm hiểu một cách hệ thống về hai giao thức quan trọng trong mạng máy tính là HTTP và HTTPS.

HTTP (HyperText Transfer Protocol) là giao thức truyền siêu văn bản không bảo mật, hoạt động theo mô hình request-response giữa client và server, sử dụng cổng TCP 80.

HTTPS (HTTP Secure) là phiên bản mở rộng và bảo mật của HTTP, sử dụng cơ chế mã hóa TLS/SSL để đảm bảo tính bảo mật, toàn vẹn và xác thực thông tin, hoạt động qua cổng TCP 443.

Thông qua việc tìm hiểu cấu trúc gói tin, nguyên lý hoạt động, và thực hiện bắt gói tin thực tế bằng Wireshark, chúng tôi nhận thấy:

HTTP cho phép truyền dữ liệu dạng plaintext (không mã hóa), khiến thông tin rất dễ bị tấn công (nghe lén, giả mạo).

HTTPS mã hóa toàn bộ nội dung truyền tải, do đó ngay cả khi bị bắt gói, hacker cũng không thể đọc được dữ liệu.

Việc triển khai HTTPS tuy có chi phí và yêu cầu kỹ thuật cao hơn, nhưng hoàn toàn cần thiết trong bối cảnh bảo mật thông tin trên Internet ngày càng trở nên cấp bách.

Như vậy, trong các hệ thống hiện đại, việc sử dụng HTTPS đã trở thành tiêu chuẩn bắt buộc đối với các website có liên quan đến giao dịch người dùng, bảo mật dữ liệu, cũng như trong lĩnh vực thương mại điện tử, tài chính, và các dịch vụ trực tuyến khác.

Định hướng học tập và mở rộng thêm:

Qua việc nghiên cứu HTTP và HTTPS, chúng tôi nhận thấy các hướng mở rộng quan trọng tiếp theo bao gồm:

Tìm hiểu HTTP/2:

HTTP/2 là phiên bản cải tiến của HTTP/1.1, hỗ trợ nén header, đa kênh (multiplexing) và push server, giúp tăng tốc độ tải trang đáng kể.

Khám phá HTTP/3 và giao thức QUIC:

HTTP/3 là thế hệ tiếp theo sử dụng giao thức QUIC (UDP-based), cải thiện độ trễ thấp, bảo mật cao và truyền dữ liệu ổn định hơn trên mạng kém.

Nghiên cứu chi tiết SSL/TLS:

Các phiên bản TLS (1.0, 1.2, 1.3).

Quá trình handshake, mã hóa đối xứng và bất đối xứng.

Các nguy cơ bảo mật như downgrade attack, certificate spoofing.

- **Ứng dụng thực tế:**

Thực hành triển khai HTTPS trên các nền tảng web server phổ biến như Apache, Nginx, IIS. Cài đặt và quản lý chứng chỉ SSL (Let's Encrypt, CA thương mại).

- **Tóm lại:**

HTTP và HTTPS là nền tảng không thể thiếu cho hoạt động của Internet ngày nay.

Việc nắm vững lý thuyết và thực hành bắt gói tin giúp sinh viên công nghệ thông tin hiểu rõ nguyên lý, phát hiện và khắc phục sự cố mạng hiệu quả hơn.

Đồng thời, kiến thức này mở ra nền tảng vững chắc cho việc tiếp cận các công nghệ mạng hiện đại như Web Security, API Protection, IoT Communications...